University Of Science And Technology Of Houari Boumediene

Faculty Of Computer Science

2nd Year Ingéniorat – Section B

UniTrans

A University Transport Management System.

Projet Finale de Base de Données 2025/2026

Analyse | Modélisation | Implémentation | Normalisation

Compiled & Developed BY:

Martin CHAUKE
23238ZWE13686
Group 1

Brighton MATIKITI
23238ZWE13691
Group 4

Supervised By:

Dr. LAHRECHE Abdelmadjid

Introduction

With the rapid advancement of information technology, web-based applications have become essential tools for managing complex systems efficiently. Universities, in particular, face increasing challenges in organizing transportation services for students due to growing demand, scheduling complexity and the need for real-time coordination.

Traditional transportation management methods often rely on manual processes, which can lead to inefficiencies, lack of transparency, and difficulties in tracking operations. To address these challenges, there is a need for a structured and automated system that ensures reliability, scalability and ease of use.

This project introduces *UniTrans*, a university transportation management system designed to streamline the management of buses, drivers, transport lines, incidents and student subscriptions. The system provides a centralized platform that allows administrators to manage transportation resources effectively while offering students a simple way to subscribe and access transport services.

In addition, the system design follows the conceptual and logical data modeling methodology, ensuring a well-structured and normalized database. This approach guarantees data integrity, reduces redundancy and facilitates future system expansion.

The objective of this project is to provide an efficient, scalable and user-friendly solution for managing university transportation systems.

System Analysis/Overview

Users of the System.

Student

Can create an account, subscribe to transport services, view assigned lines and schedules, and manage their subscription.

Driver

Can view assigned trips, monitor schedules, and report incidents occurring during service.

Transport Manager / Administrator

Has full control over the system including: Managing students, managing drivers, managing buses, managing lines and stations, creating schedules, assigning buses and drivers, monitoring subscriptions, reviewing incidents, and managing line allocations.

UniTrans – University Bus Transport Management System

System Architecture

FRONTEND LAYER

1. MANAGER PORTAL (Next.js) – Port 3000
Admin / Transport Staff Interface
Tech: React, TypeScript, Axios, Context API

- Dashboard
- Students Management
- Buses Management
- Drivers Management
- Lines Management
- Trips Management
- Incidents Management
- Subscriptions Management
- Statistics & Analytics
- Reports Generation

2. STUDENT PORTAL (Next.js) – Port 3001
Student Self-Service Interface
Tech: React, TypeScript, Axios, Context API

- My Assigned Transport
- My Schedules
- My Subscriptions
- Subscription History
- Request Line Change
- My Notifications
- Profile Management
- Announcements & Alerts
- Help & Support

3. DRIVER PORTAL (Next.js) – Port 3002
Driver / Field Operations Interface
Tech: React, TypeScript, Axios, Context API

- Today's Trips
- Assigned Bus
- Route / Line Details
- Trip Status Start / Complete / Delay
- Passenger / Seat Context
- Incidents Reporting
- Notifications (Dispatch / Changes)
- Profile Management
- Help & Support

HTTPS / REST API (JSON)

LocalStorage (Manager Namespace)
access_token_manager
refresh_token_manager

LocalStorage (Student Namespace)
access_token_student
refresh_token_student

LocalStorage (Driver Namespace)
access_token_driver
refresh_token_driver

BACKEND LAYER

DJANGO REST FRAMEWORK API – Port 8000

JWT Authentication
- Access / Refresh Tokens
- Refresh Rotation
- Token Blacklist (Logout)

CORS
Enabled for Origins: :3000, :3001, :3002

Permissions & Role-Based Access
(Manager / Student / Driver)

API Documentation
- Swagger UI
- ReDoc

Global Exception Handling & Logging
- Request / Response Logs
- Error Tracking

DJANGO APPLICATIONS (9)

1. Accounts
- Registration
- Authentication
- User (Role)
- Student Profile
- Driver Profile
- JWT Management

2. Lines
- Routes (Lines)
- Stations
- Line–Station Mapping

3. Schedules
- Timetables
- Schedule Definitions

4. Buses
- Fleet Management
- Bus–Line Assignments
- Driver Linkage (where applicable)

5. Trips
- Trip Planning
- Trip Execution
- Rows
- Seat Assignments
- Trip Status
- Driver (Trip)

6. Subscriptions
- Student Subscriptions
- Subscription History
- Renewal Tracking
- Audit Trail

7. Incidents
- Report Incidents
- Track Issues
- Status Updates
- Reported By: Student / Driver

8. Notifications
- Alerts & Messages
- Triggers From:
  - Subscriptions
  - Incidents
  - Trip Updates
- Delivery Rules to Students & Drivers

9. Reports
- Analytics
- Aggregation From:
  - Trips
  - Subscriptions
  - Incidents
  - Driver / Trip KPIs

Core DRF Components
Serializers (JSON) • ViewSets / API Endpoints • Filters / Pagination • Throttling • Permissions

INTEGRATIONS / SERVICES
- Email Service (SMTP)
- SMS Gateway
- Push Notifications
- File Storage (MinIO / S3 Compatible)
- Logging & Monitoring (Prometheus / Grafana)

DATABASE LAYER

PostgreSQL 14+ Database (Single Database for All Apps)

users
PK user_id
name
email (unique)
password
role (manager/ student/driver)
is_active
is_staff
date_joined

students
PK student_id
FK user_id
registration_number (unique)
first_name
last_name
email
phone

drivers
PK driver_id
FK user_id
license_number (unique)
first_name
last_name
phone
assigned_bus_id (FK)

lines
PK line_id
name
description
is_active

stations
PK station_id
name
location
latitude
longitude

line_stations
PK line_station_id
FK line_id
FK station_id
order_index
UNIQUE (line_id, station_id)

buses
PK bus_id
registration_number (unique)
model
capacity
status

bus_assignments
PK bus_assignment_id
FK bus_id
FK line_id
start_date
end_date
notes

schedules
PK schedule_id
FK line_id
day_of_week
departure_time
arrival_time
direction

trips
PK trip_id
FK schedule_id
FK bus_id
FK driver_id (nullable)
actual_departure
actual_arrival
status

rows
PK row_id
FK bus_id
row_number
seat_count
UNIQUE (bus_id, row_number)

seat_assignments
PK seat_assignment_id
FK trip_id
FK row_id
FK student_id
seat_number
assigned_at
UNIQUE (trip_id, row_id, seat_number)

subscriptions
PK subscription_id
FK student_id
FK line_id
start_date
end_date
is_active

subscription_histories
PK subscription_history_id
FK student_id
FK old_line_id
FK new_line_id
change_date
reason

incidents
PK incident_id
FK trip_id (nullable)
reported_by_type (student/driver)
reported_by_id (nullable)
title
description
status
reported_at
resolved_at

notifications
PK notification_id
FK user_id
notification_type
title
message
is_read
created_at
related_entity_type
related_entity_id

student_reports
PK report_id
FK student_id
report_type
title
description
status
submitted_at
resolved_at

report_aggregates
PK aggregate_id
metric_type
period_start
period_end
value
generated_at
generated_by

DATA FLOW (REQUEST / RESPONSE)

1. User logs in via any portal (Manager / Student / Driver).
2. Credentials sent to Auth API.
3. API validates and returns JWT (access + refresh).
4. Frontend stores tokens in portal-specific localStorage.
5. Subsequent requests include:
   Authorization: Bearer <access>
6. API validates JWT, checks role (manager / student / driver) and permissions.
7. Business logic executed via Django ORM.
8. JSON response serialized and returned; UI renders data.

AUTHENTICATION FLOW

1. Login request → returns access + refresh tokens.
2. Access token used for API requests.
3. When expired → frontend sends refresh token to refresh endpoint.
4. New access (and rotated refresh) token returned.
5. On logout → refresh token is blacklisted.
6. Each portal uses its own localStorage namespace:
   - Manager: access_token_manager / refresh_token_manager
   - Student: access_token_student / refresh_token_student
   - Driver: access_token_driver / refresh_token_driver
   → Allows concurrent logins without collision.

KEY RELATIONSHIPS

- User → Student & Driver (profile children).
- Line → Station, Timetable (Schedules).
- Bus → BusAssignment → Line.
- Bus → Trip.
- Trip → Row → SeatAssignment → Student.
- Trip ↔ Driver (operational assignment).
- Student → Subscription → Line.
- Student → Incident / Notification.
- Driver → Trip / Bus (as assigned).
- Driver → Notification.
- Reports aggregate Trip, Subscription, Incident (and Driver / Trip KPIs).

LEGEND

- Component / Module
- Use Case / Feature
- Database / Storage
- Synchronous (HTTPS / REST)
- Async / Event Trigger (Notifications, Jobs)
- 1 — One
- 1..* — One to Many
- 0..* — Zero to Many
- Aggregation / Derived

Figure 1: Global System Architecture of Unitrans

Business Rules (Régles des contraintes et Intégrités).

1. A student subscribes to a transport line and not directly to a bus.
2. A student can have multiple subscriptions overs time
3. Each subscription links one student to one transport line.
4. SubscriptionHistory records subscription modifications, renewals, and cancellations.
5. A transport line serves multiple stations in a defined order.
6. A transport line can have multiple buses assigned over time.
7. A bus can be assigned to different transport lines over time.
8. A bus contains multiple rows.
9. Each row contains multiple seats.
10. Drivers can be assigned to different buses overtime.(But a driver can drive one bus only for the assigned period and can change to be assigned to another).
11. A bus can have different drivers over time.
12. Each trip is generated from a schedule.
13. A bus can perform multiple trips.
14. Each trip is executed by one driver and one bus.
15. Incidents are associated with a specific trip.
16. Students can be assigned seats for specific trips.
17. Users can have one of the following roles: Admin, Transport Manager, Student and Driver.

Conceptual Data Model(MCD): Modele-Entite Association.

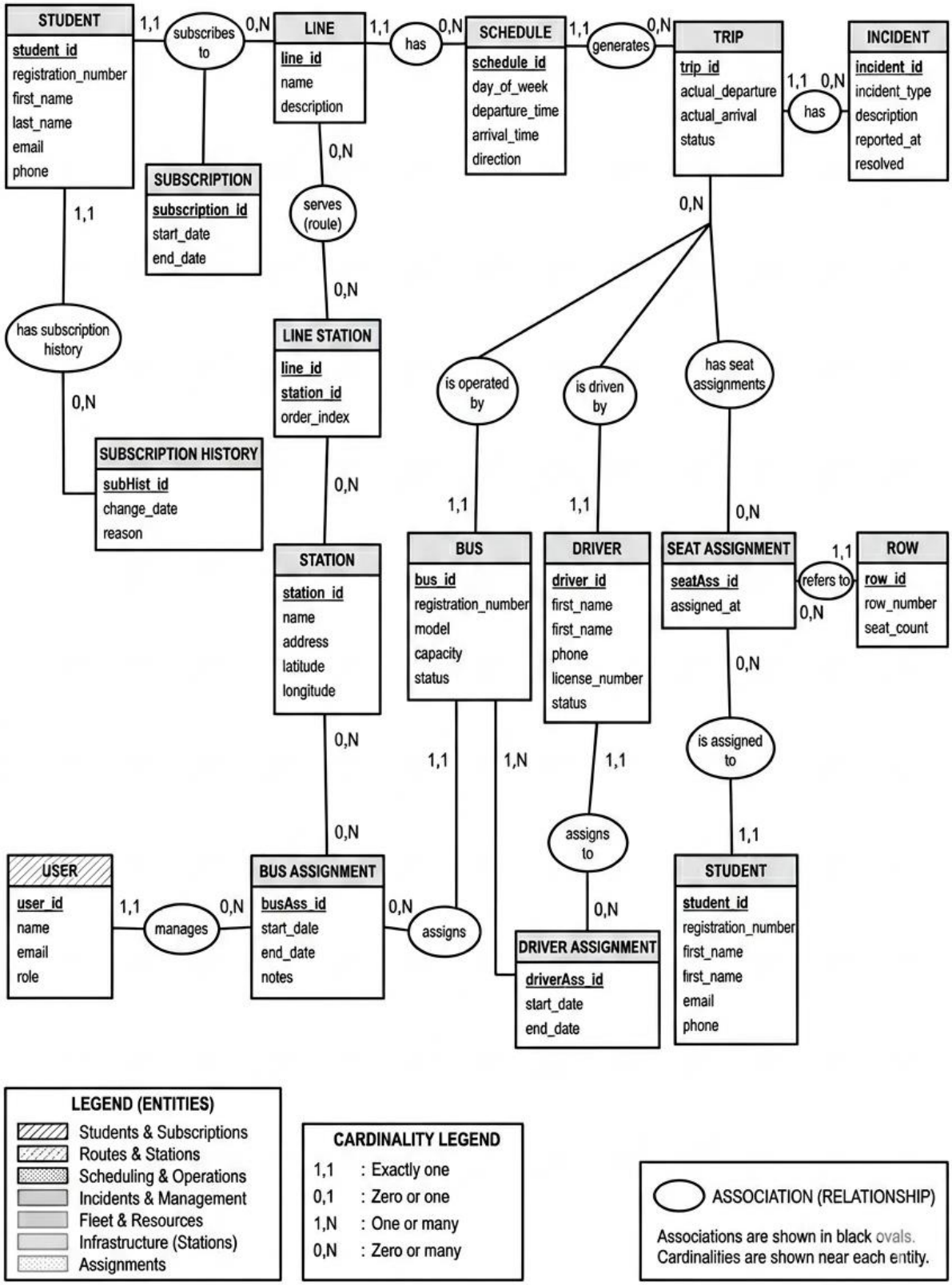
The Entities.

1. *Student*: Represents a university student using transport services.
2. *Driver:* Represents a driver assigned to operate buses.
3. *User:* Represents authenticated system users.

   Roles:

   Admin, Transport Manager, Student and Driver.
4. *Bus:* Represents a vehicle that can be assigned to transport lines over time.
5. *Row*: Represents seat rows inside a bus.
6. *Line*: Represents a transport route.
7. *Station:* Represents stops served by transport lines.
8. *Schedule*: Represents planned departure and arrival times for a line.
9. *Trip:* Represents an actual transport execution generated from a schedule.
10. *Subscription*: Represents student registration to a transport line.
11. *SubscriptionHistory*: Represents modifications, renewals, and cancellations of subscriptions.
12. *Incident*: Represents problem reported during a trip.
13. *BusAssignment*: Represents temporal assignment of buses to transport lines.
14. *DriverAssignment:* Represents temporal assignment of drivers to buses.
15. *SeatAssignment*: Represents student seat allocation during trips.
16. *SERVES*: Associative entity linking lines and stations with route order.

Figure 2: Conceptual Data Model of Unitrans.



Relational Model/Modéle-Relationele De Données(MLD).

User(*user_id*, name, email, role)

Student(*student_id*, registration_number, first_name, last_name, email, phone, *#user_id*)

Driver(*driver_id*, first_name, last_name, phone, license_number, status, *#user_id*)

Line(*line_id*, name, description)

Station(*station_id*, name, address, latitude, longitude)

Bus(*bus_id,* registration_number, model, capacity, status)

Row(*row_id*, row_number, seat_count, *#bus_id*)

Schedule(*schedule_id*, day_of_week, departure_time, arrival_time, direction, *#line_id*)

Trip(*trip_id,* actual_departure, actual_arrival, status, *#schedule_id*, *#bus_id*, *#driver_id*)

Incident(incident_id, incident_type, description, reported_at, resolved, *#trip_id*)

Subscription(subscription_id, start_date, end_date, *#student_id*, *#line_id*)

SubscriptionHistory(*subscription_history_id*, change_date, action_type, reason, *#student_id*, *#line_id*)

BusAssignment(bus_assignment_id, start_date, end_date, notes, *#bus_id*, *#line_id*, *#user_id*)

DriverAssignment(*driver_assignment_id,* start_date, end_date, *#driver_id*, *#bus_id*)

SeatAssignment(*seat_assignment_id*, assigned_at, *#student_id*, *#trip_id*, *#row_id*)

SERVES*(#line_id*, *#station_id*, order_index)

Note: The attribute student_id is used as the primary key instead of the registration_number. This design ensures a stable, system generated unique identifier that is independent of external or administrative changes, while the registration_number remains a unique but non-primary attribute.

IMPLEMENTATION SCOPE

Primary Keys.

Each entity contains a unique identifier:

- user_id
- student_id
- driver_id
- line_id
- station_id
- bus_id
- row_id
- schedule_id
- trip_id
- incident_id
- subscription_id
- subscription_history_id
- bus_assignment_id
- driver_assignment_id
- seat_assignment_id

Foreign Keys

Labeled blue in the relational model.(refer up.)

Normalization

The Unitrans database satisfies:

First Normal Form(1NF): All attributes are atomic.

Second Normal Form(2NF): All non-key attributes depend fully on the primary key and there are no partial functional dependences.

Third Normal Form (3NF): No transitive dependences exist. For example: Driver information depends only on *driver_id*, Subscription status depends only on *subscription_id,* and Bus assignment data depends only on *bus_assignment_id*.

BCNF: Every functional dependence has a candidate key as the determinant.

APPLICATION DESCRIPTION AND CONCEPTION

Manager/Admin Portal

Handles user management, Student management, Driver management, Bus assignment, Driver assignment, Schedule management, Incident review and Statistics.

Student Portal

Handles registration/Login, Subscription, Subscription renewal, Seat allocation and Schedule viewing.

Driver Portal

Handles assigned trips and bus, Route/Line details and Incident reporting.

Use case diagram

Summarizes the application flow and functionality.

Figure 3: Use case diagram

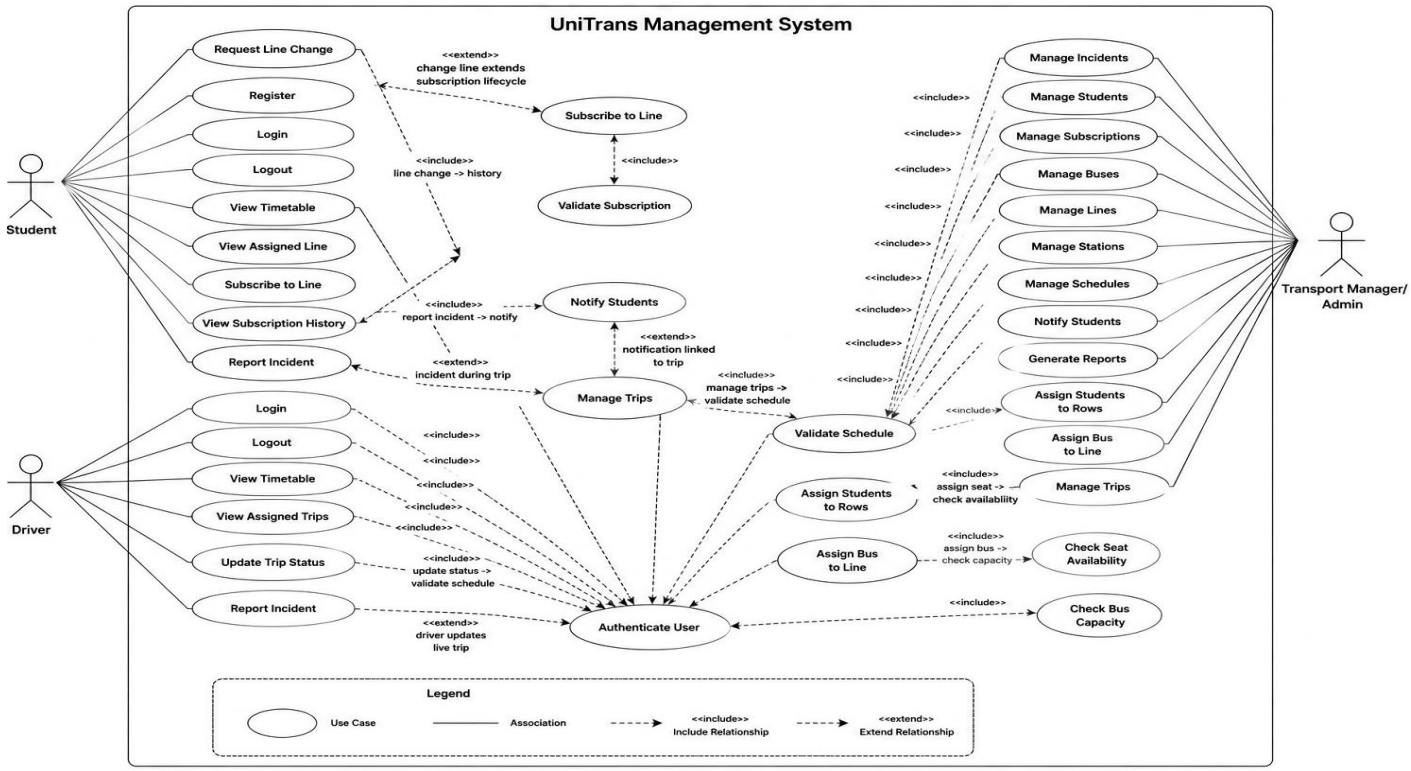
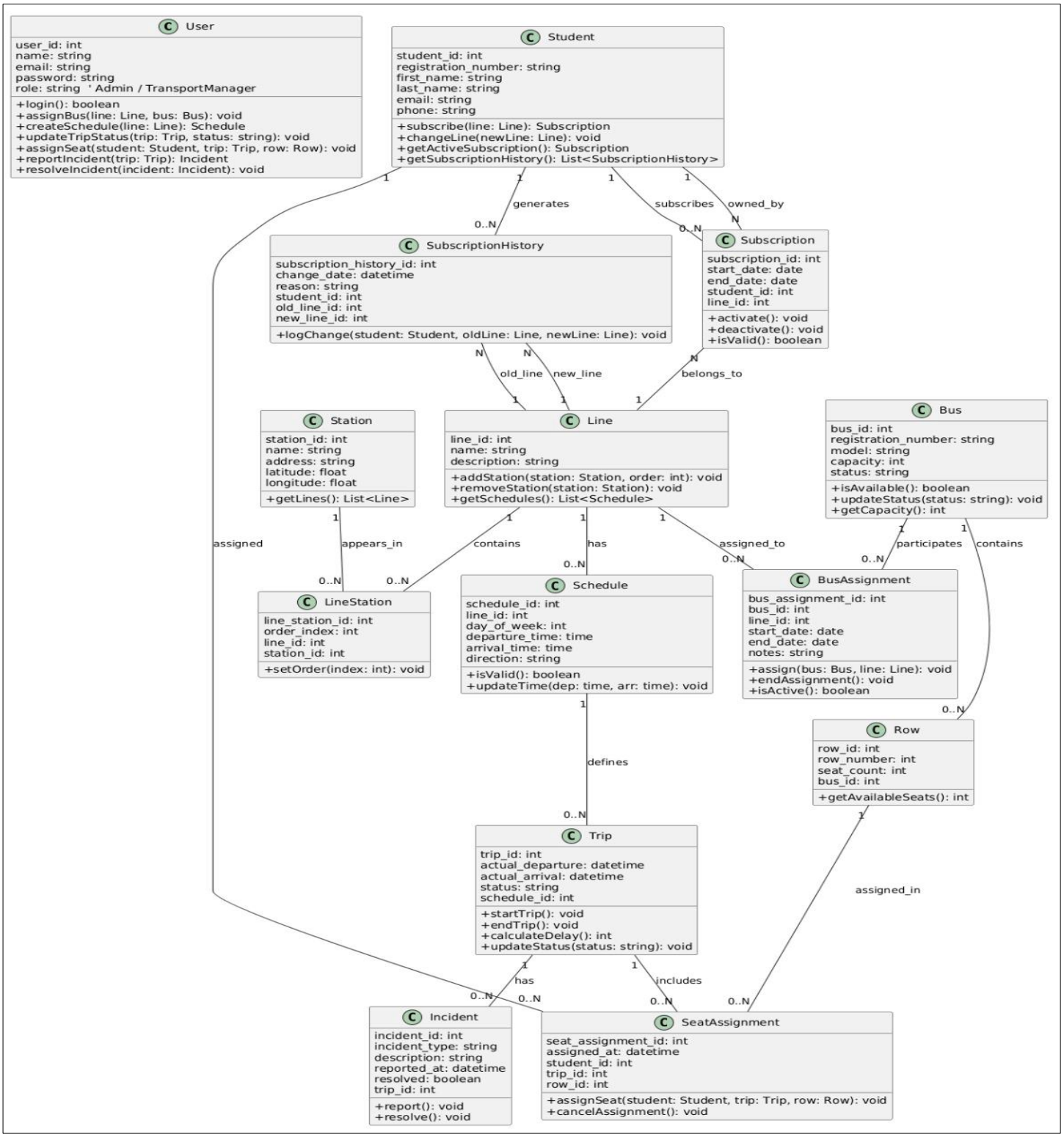


Figure 4: Class diagram of Unitrans App



APPLICATION INTERFACES

Authentication Interfaces.

Student

Figure 5: Student landing page

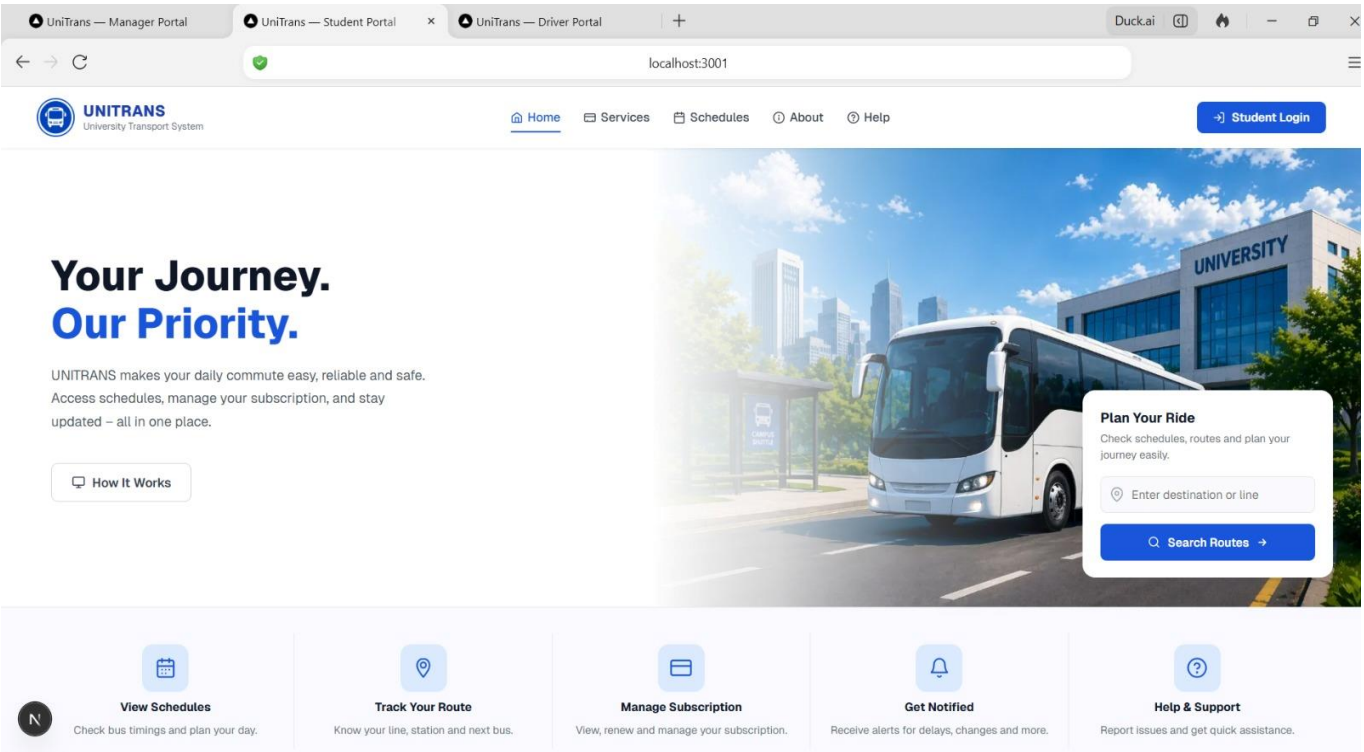


Figure 6: Student Login

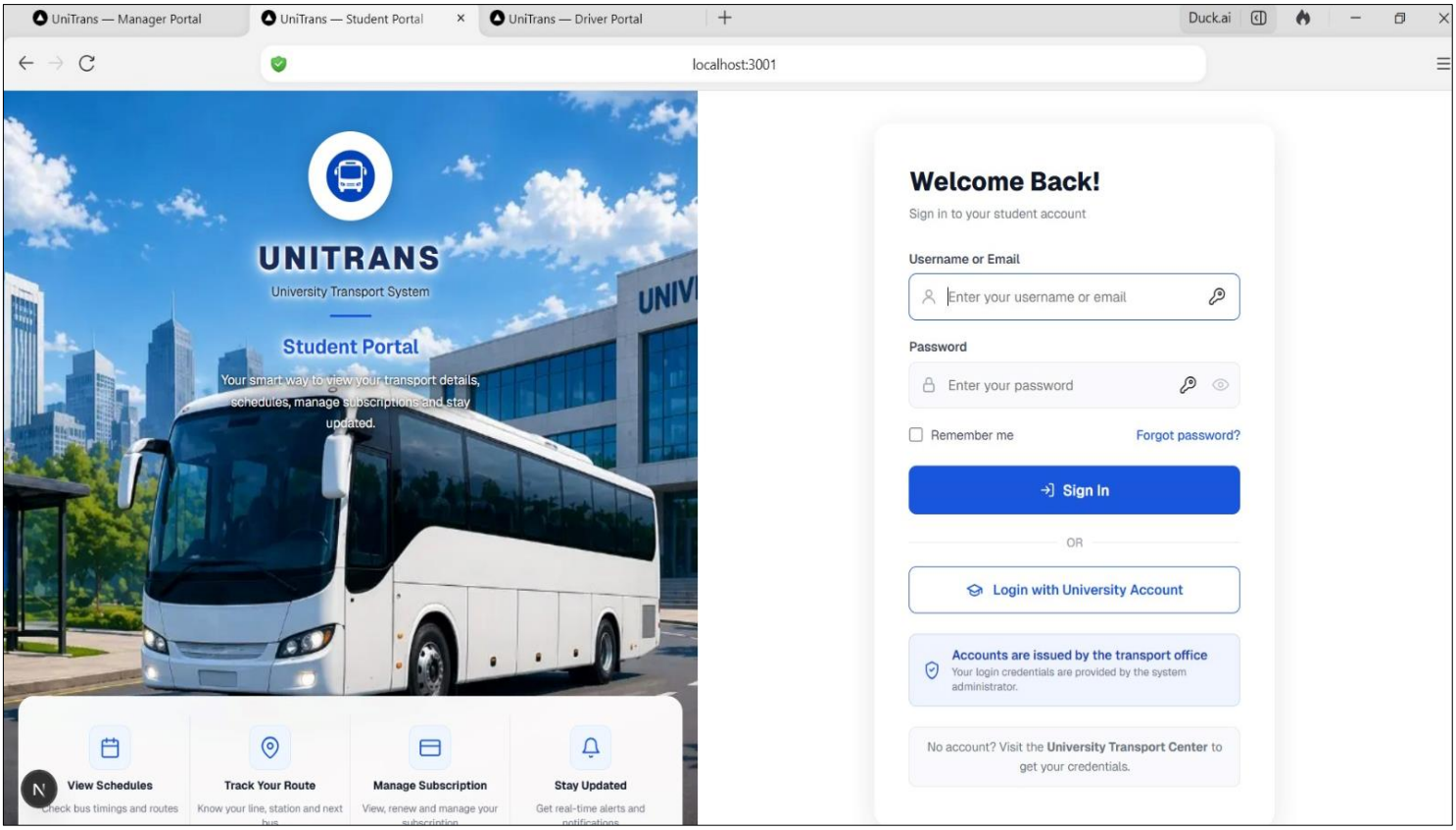
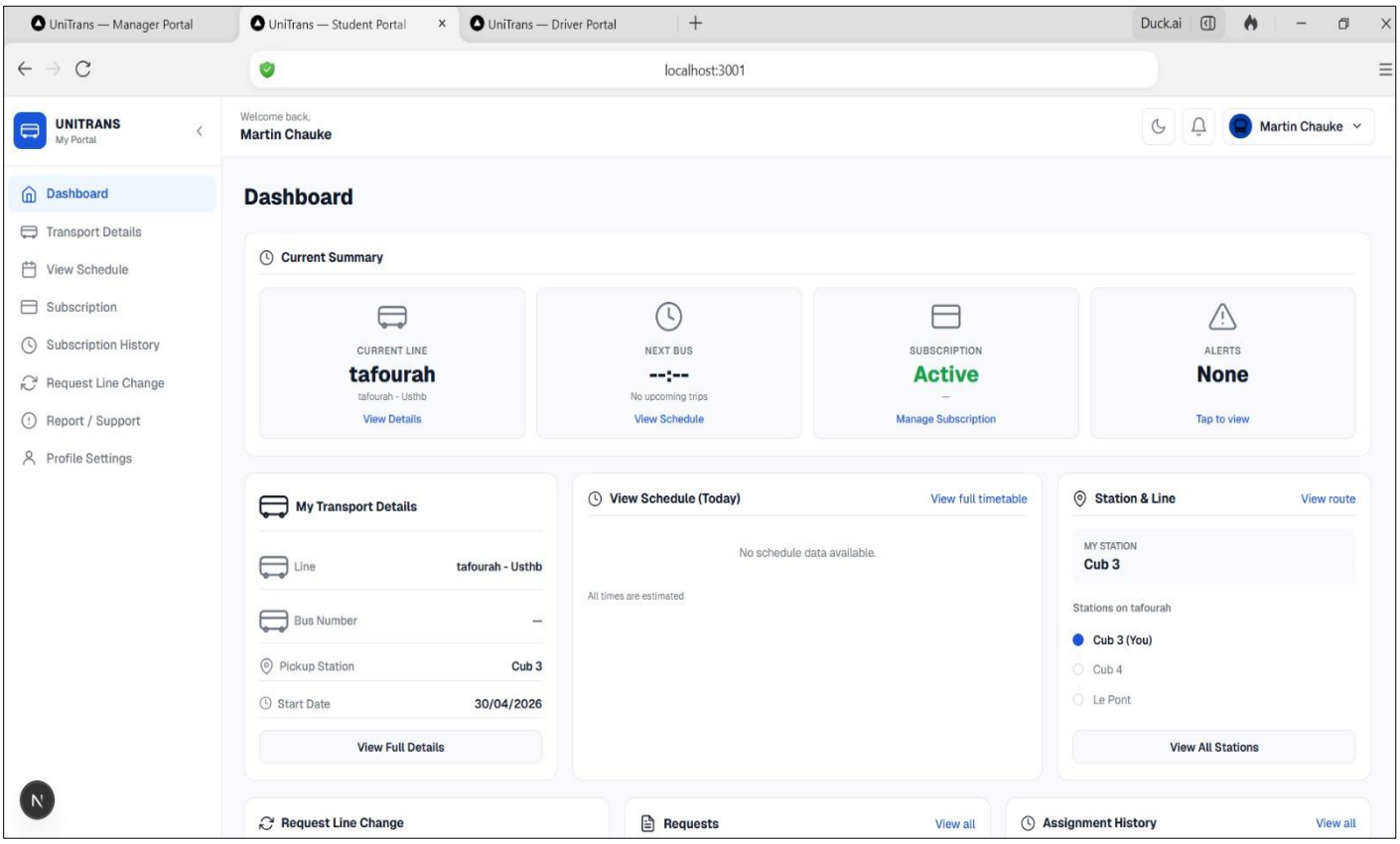


Figure 7: Student Dashboard



Driver

The Driver Dashboard provides drivers with access to their assigned trips and daily schedules. It allows them to view route details and transport instructions in real time. Drivers can also report incidents directly through the system for quick administrative response.

Figure 8: Driver Landing Page

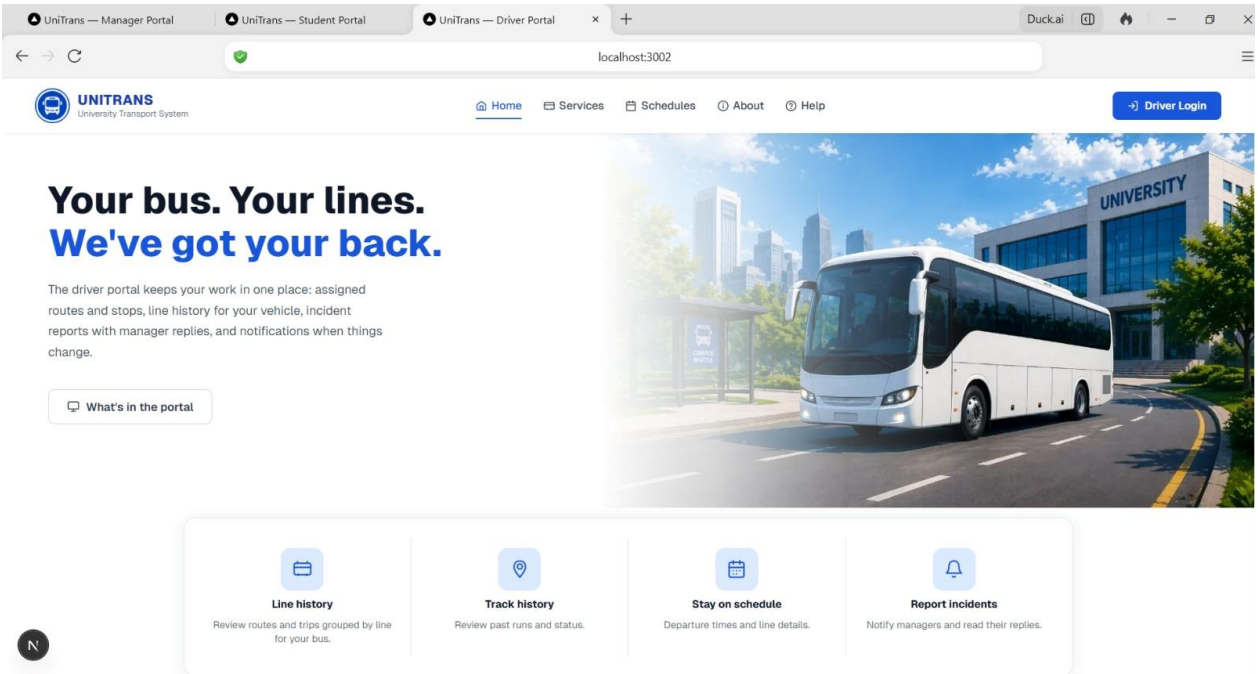


Figure 9: Driver Login

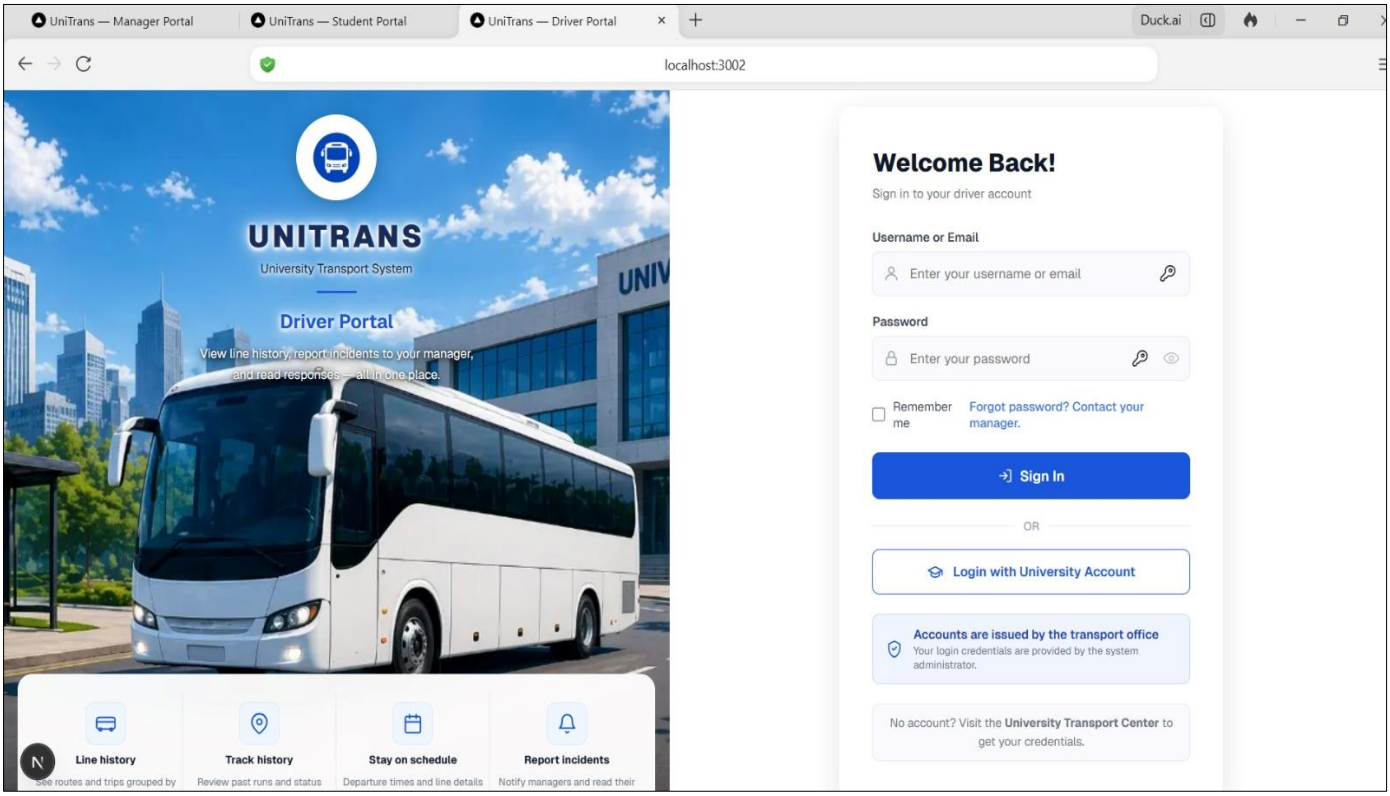
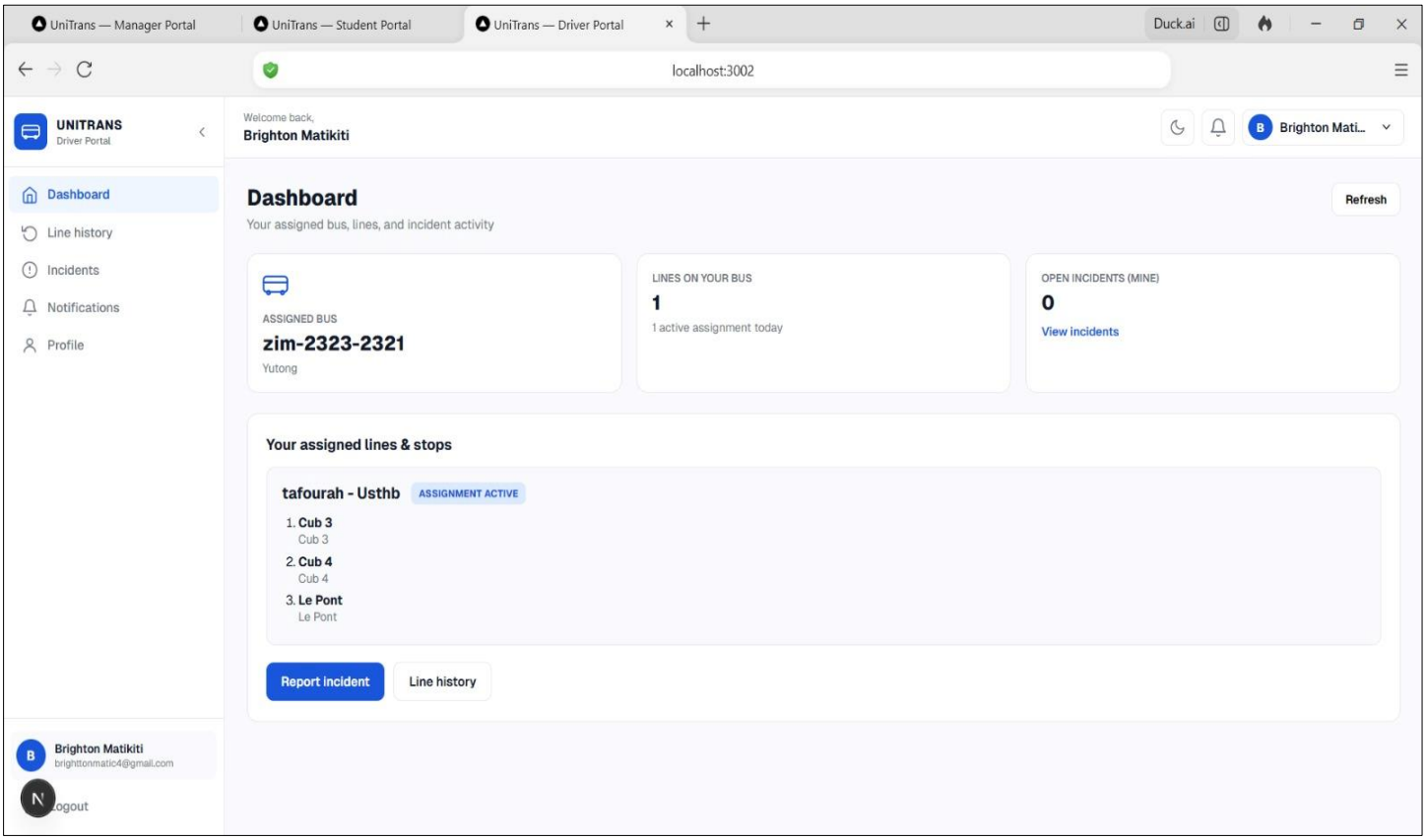


Figure 10: Driver Dashboard



Manager

Figure 11: Manager Landing Page

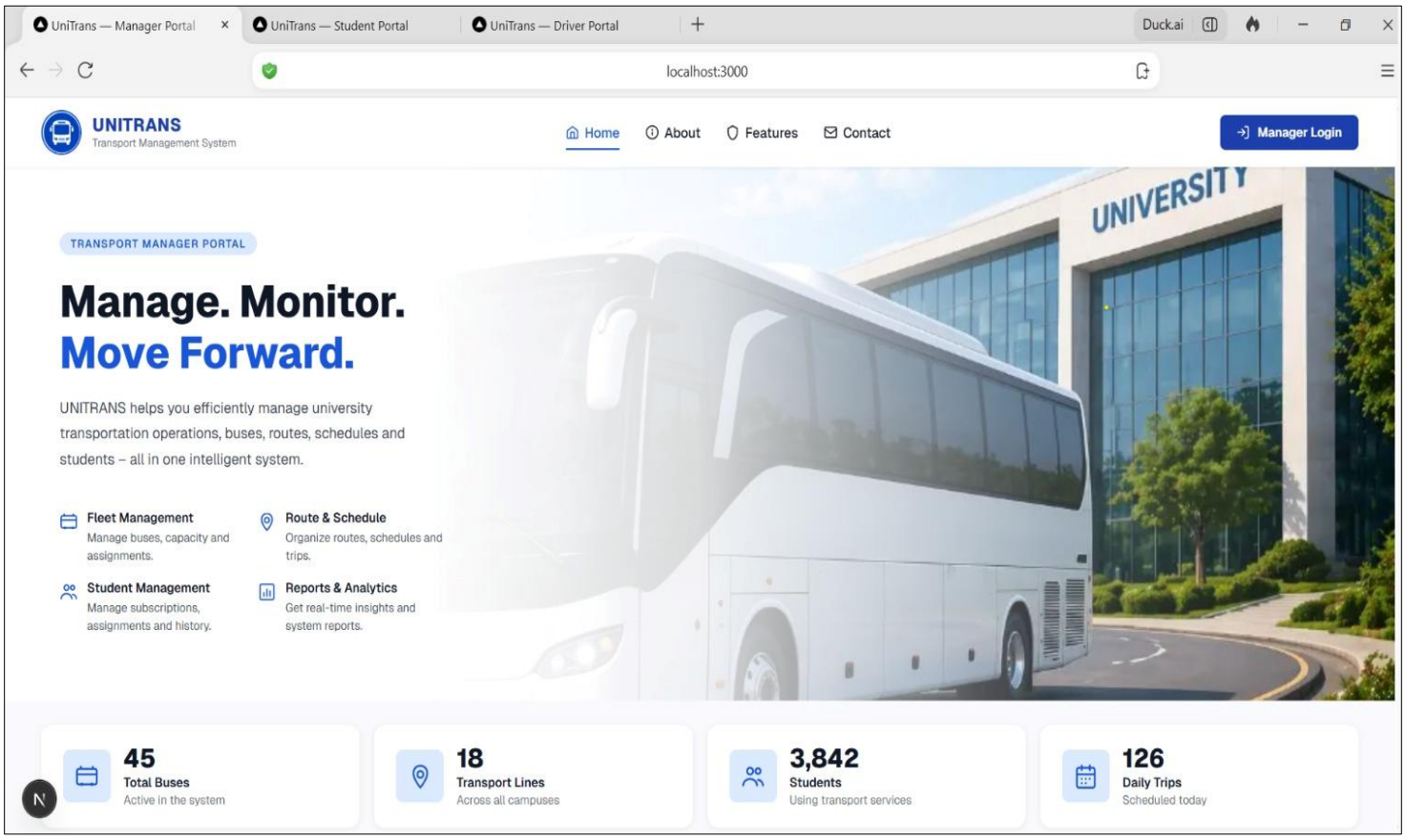


Figure 12: Manager Login

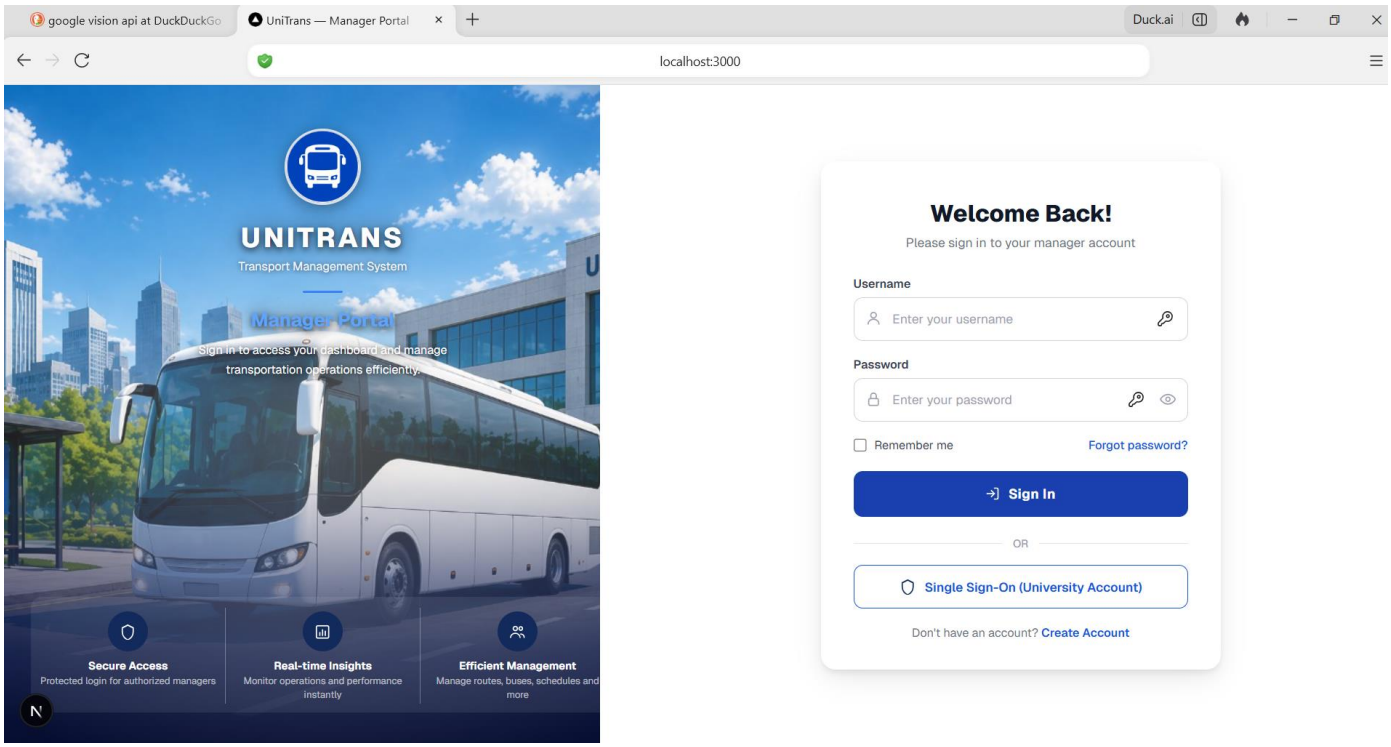
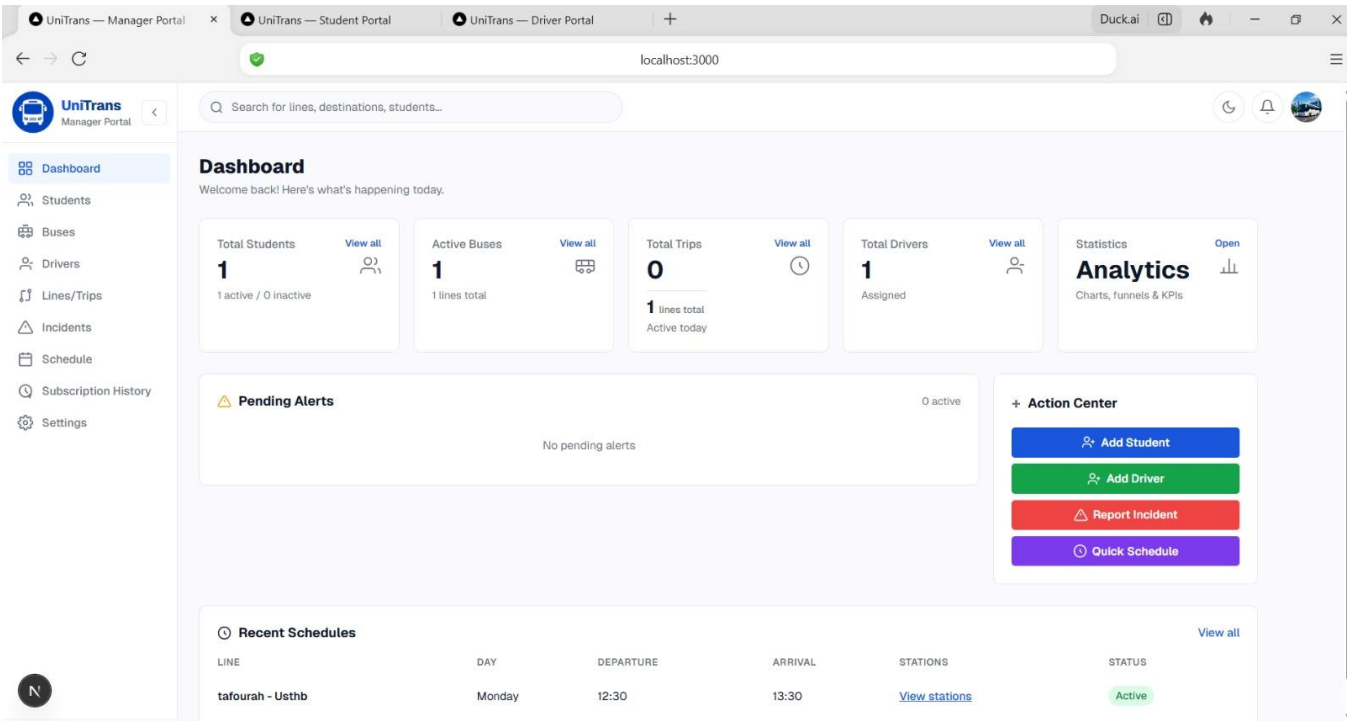


Figure 13: Manager's Dashboard

The Manager Dashboard provides a centralized interface for monitoring transport operations, managing assignments, and accessing key system statistics efficiently.



Statistics

The statistics section provides a real-time overview of the transport system's performance and usage. It displays key indicators such as the number of active students, buses in service, and ongoing trips. This helps the Transport Manager monitor system efficiency and identify operational issues quickly. It also supports data-driven decision-making for improving service quality and resource allocation.

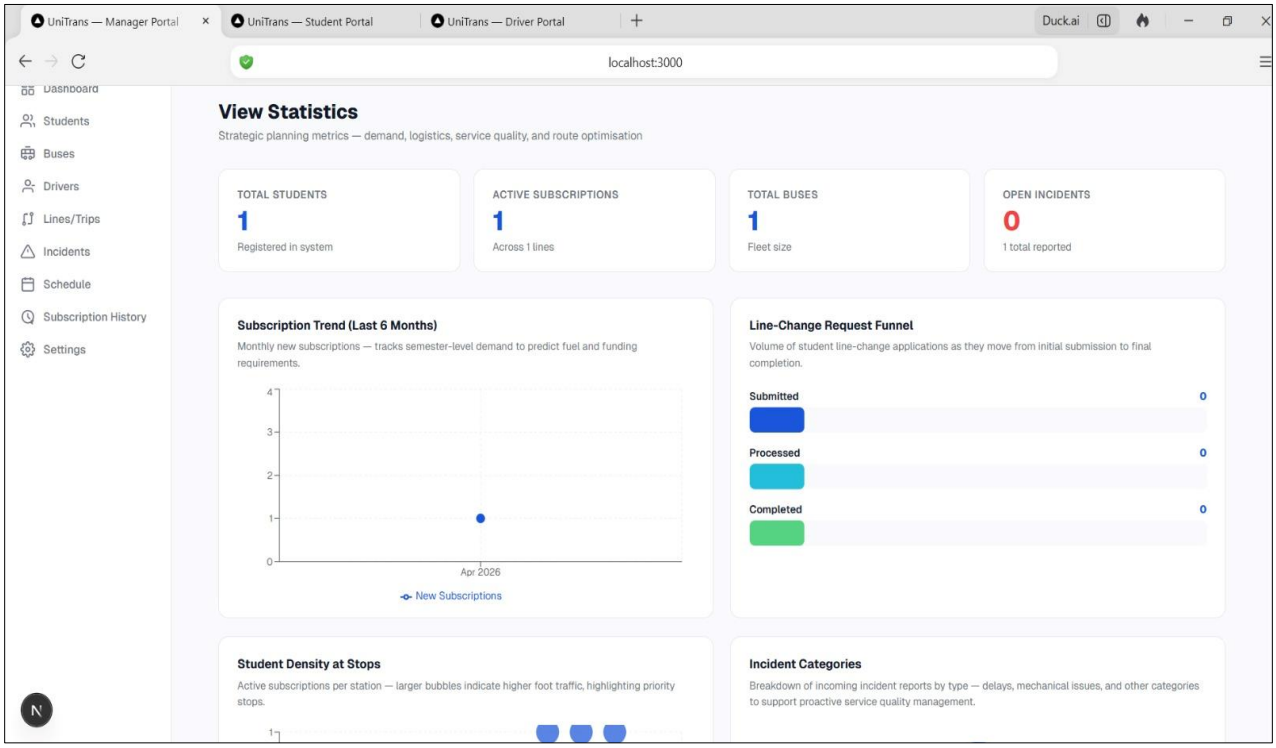
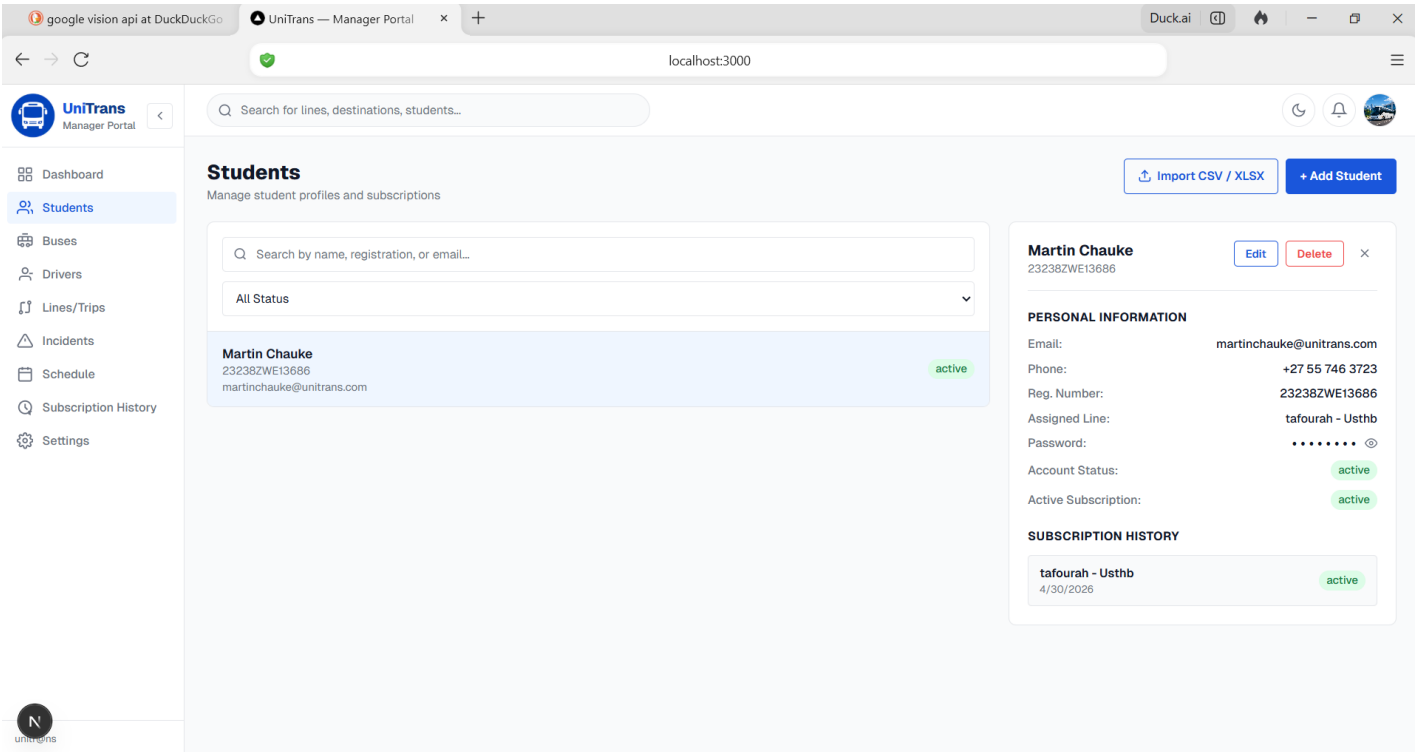


Figure 14: System Statistics Interface

Figure 15: View Students by Manager.



PostrgreSQL database Schema

SQL tables creation.

NB: *student_id* is used as the primary key because it provides a stable system-generated identifier, while *registration_number* remains a unique administrative reference

User.

```
CREATE TABLE "User" (
    user_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (
        role IN ('Admin','TransportManager','Student','Driver')  )
```

```
);
```

Student

```
CREATE TABLE Student (
    student_id SERIAL PRIMARY KEY,
    registration_number VARCHAR(50) UNIQUE NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    phone VARCHAR(20),
    user_id INTEGER REFERENCES "User"(user_id)
);
```

Driver

```
CREATE TABLE Driver (
    driver_id SERIAL PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    license_number VARCHAR(50) UNIQUE NOT NULL,
    status VARCHAR(20) DEFAULT 'active',
    user_id INTEGER REFERENCES "User"(user_id)
);
```

Line

```
CREATE TABLE Line (
    line_id SERIAL PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL,
    description TEXT
);
```

Station

```
CREATE TABLE Station (
    station_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    address VARCHAR(255),
    latitude FLOAT,
    longitude FLOAT
);
```

Bus

```
CREATE TABLE Bus (
    bus_id SERIAL PRIMARY KEY,
    registration_number VARCHAR(50) UNIQUE NOT NULL,
    model VARCHAR(100),
    capacity INTEGER NOT NULL,
    status VARCHAR(20) DEFAULT 'active'
);
```

Row

```
CREATE TABLE Row (
```

```
    row_id SERIAL PRIMARY KEY,
    row_number INTEGER NOT NULL,
    seat_count INTEGER NOT NULL,
    bus_id INTEGER NOT NULL REFERENCES Bus(bus_id) ON DELETE CASCADE
);
```

Schedule

```
CREATE TABLE Schedule (
    schedule_id SERIAL PRIMARY KEY,
    day_of_week VARCHAR(10) NOT NULL,
    departure_time TIME NOT NULL,
    arrival_time TIME NOT NULL,
    direction VARCHAR(20),
    line_id INTEGER NOT NULL REFERENCES Line(line_id)
);
```

Trip

```
CREATE TABLE Trip (
    trip_id SERIAL PRIMARY KEY,
    actual_departure TIMESTAMP,
    actual_arrival TIMESTAMP,
    status VARCHAR(20) DEFAULT 'scheduled',
    schedule_id INTEGER REFERENCES Schedule(schedule_id),
```

```
    bus_id INTEGER REFERENCES Bus(bus_id),
    driver_id INTEGER REFERENCES Driver(driver_id)
);
```

Incident

```
CREATE TABLE Incident (
    incident_id SERIAL PRIMARY KEY,
    incident_type VARCHAR(100),
    description TEXT,
    reported_at TIMESTAMP DEFAULT NOW(),
    resolved BOOLEAN DEFAULT FALSE,
    trip_id INTEGER NOT NULL REFERENCES Trip(trip_id)
);
```

Subscription

```
CREATE TABLE Subscription (
    subscription_id SERIAL PRIMARY KEY,
    start_date DATE NOT NULL,
    end_date DATE,
    student_id INTEGER NOT NULL REFERENCES Student(student_id),
    line_id INTEGER NOT NULL REFERENCES Line(line_id)
);
```

SubscriptionHistory

```
CREATE TABLE SubscriptionHistory (
    subscription_history_id SERIAL PRIMARY KEY,
    change_date TIMESTAMP DEFAULT NOW(),
    action_type VARCHAR(30),
    reason TEXT,
    student_id INTEGER REFERENCES Student(student_id),
    line_id INTEGER REFERENCES Line(line_id)
);
```

BusAssignment

```
CREATE TABLE BusAssignment (
    bus_assignment_id SERIAL PRIMARY KEY,
    start_date DATE NOT NULL,
    end_date DATE,
    notes TEXT,
    bus_id INTEGER NOT NULL REFERENCES Bus(bus_id),
    line_id INTEGER NOT NULL REFERENCES Line(line_id),
    user_id INTEGER REFERENCES "User"(user_id)
);
```

DriverAssignment

```
CREATE TABLE DriverAssignment (
    driver_assignment_id SERIAL PRIMARY KEY,
```

```
    start_date DATE NOT NULL,
    end_date DATE,
    driver_id INTEGER NOT NULL REFERENCES Driver(driver_id),
    bus_id INTEGER NOT NULL REFERENCES Bus(bus_id)
);
```

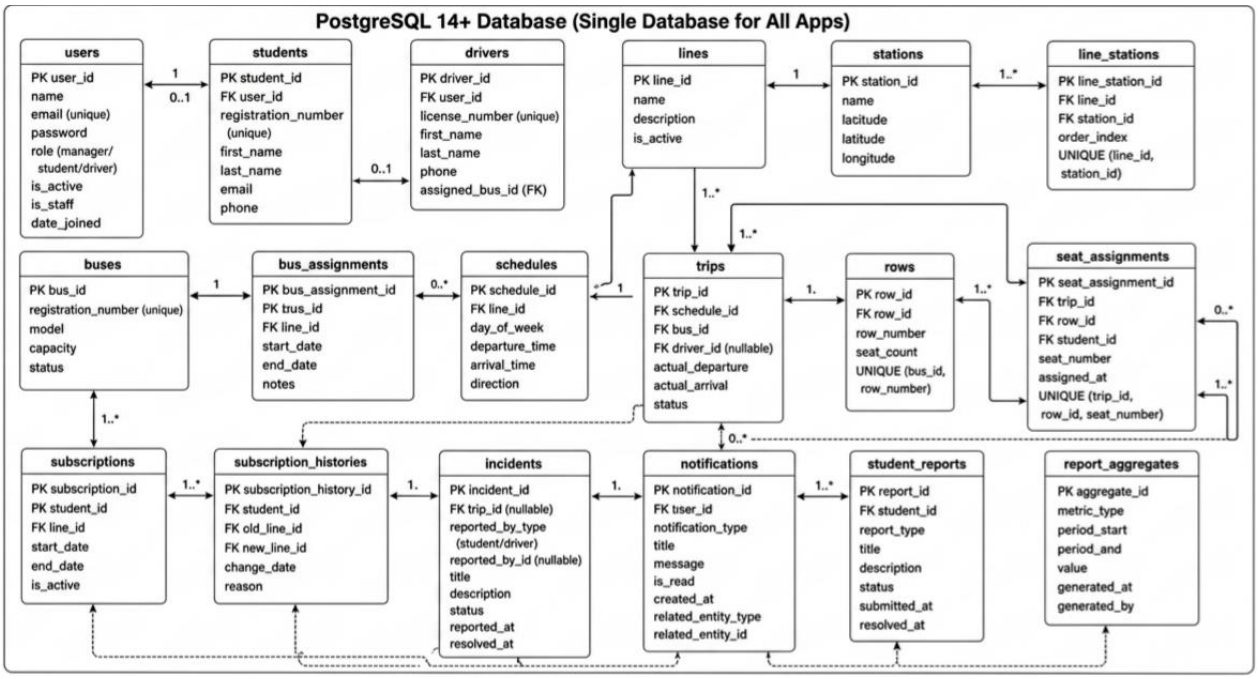
SeatAssignment

```
CREATE TABLE SeatAssignment (
    seat_assignment_id SERIAL PRIMARY KEY,
    assigned_at TIMESTAMP DEFAULT NOW(),
    student_id INTEGER NOT NULL REFERENCES Student(student_id),
    trip_id INTEGER NOT NULL REFERENCES Trip(trip_id),
    row_id INTEGER NOT NULL REFERENCES Row(row_id)
);
```

SERVES

```
CREATE TABLE SERVES (
    line_id INTEGER NOT NULL REFERENCES Line(line_id),
    station_id INTEGER NOT NULL REFERENCES Station(station_id),
    order_index INTEGER NOT NULL,
    PRIMARY KEY (line_id, station_id)
);
```

Figure 16: Database Schema



Conclusion

The *UniTrans* system was designed to provide an efficient and structured solution for managing university transportation services. The main objective of this project was to replace manual and unorganized transport management with a centralized database system.

Through the analysis and design phases, the system was modeled using conceptual design (MCD), relational model (MLD), and normalized up to the BCNF, ensuring data consistency and reducing redundancy.

The implementation of the system allows better management of students, transport lines, buses, drivers, subscriptions, and incidents.

GitHub Repository: *UniTrans_Source_Code*

References.

[1] Silberschatz, A., Korth, H. F., Sudarshan, S.
[2] Elmasri, R., Navathe, S. B.
[3] Date, C. J.
[4] Connolly, T., Begg, C.
[5] PostgreSQL Documentation
[6] Oracle SQL Language Reference Manual.